Lecture Notes: TwinCAT 3

Version Control in TwinCAT 3

March 14, 2026

Contents

1 Introduction to Version Control **2**
- 1.1 The "Old Way" . . . 2
- 1.2 Problems with Manual Folders . . . 2
- 1.3 Benefits of VCS . . . 2

2 Git: Distributed Version Control **2**
- 2.1 Key Components . . . 3
- 2.2 The Standard Workflow . . . 3

3 TwinCAT 3 Specifics **4**
- 3.1 The .gitignore File . . . 5
- 3.2 Essential Settings for Clean Code History . . . 6
  - 3.2.1 1. Separate LineIDs . . . 6
  - 3.2.2 2. Independent Project Files . . . 6

4 Visual Comparisons (Diffing) **8**

5 Conclusion **9**
- 5.1 Summary of Best Practices . . . 9
- 5.2 Quick Start Checklist . . . 9

Quick Reference Card **9**

1 Introduction to Version Control

Version control is the practice of tracking and managing changes to software code over time. For automation engineers, using a version control system (VCS) is a mark of **professionalism**, regardless of the project's size.

1.1 The "Old Way"

Historically, engineers managed revisions by manually copying folders and renaming them (e.g., "Machine_Version_0.8," "Machine_Release_Final").

1.2 Problems with Manual Folders

This leads to "version madness," where it is impossible to track what is installed on a machine, making collaboration between multiple developers nearly impossible and highly prone to error.

Folder Name	Problem
`Project_Final`	Which final?
`Project_Final_v2`	Is this newer?
`Project_REALLY_Final`	Confusion increases
`Project_Final_John_Edit`	Who has the latest?
`Project_USE_THIS_ONE`	Chaos ensues

Figure 1: The "Version Madness" Problem

1.3 Benefits of VCS

- **Long-term History:** Tracks every change, including who made it, when, and why (via commit notes).
- **Root Cause Analysis:** Enables developers to revert to previous versions to identify when a bug was introduced.
- **Branching and Merging:** Allows individuals or teams to work on separate features in independent streams before merging them back together.
- **Traceability:** Connects code changes to project management and bug-tracking software.

Aspect	Manual Folders	Version Control
History	Lost or unclear	Complete and searchable
Collaboration	Error-prone	Structured and safe
Reverting	Manual copy/paste	One command
Blame/Attribution	Unknown	Tracked per line
Branching	Copy entire folder	Lightweight branches

Table 1: Manual Folders vs. Version Control

2 Git: Distributed Version Control

Git is the world's most popular version control system. It is a **distributed** system, meaning every user has a full copy of the project history on their own hard drive, rather than relying solely on a central server.

2.1 Key Components

1. **Working Directory:** The actual project folder on your computer.
2. **Stage (Index):** A middle ground where you mark changes to be included in the next "snapshot".
3. **Local Repository:** The hidden `.git` folder on your drive that stores the full history of snapshots (commits).
4. **Remote Repository:** A version of the project hosted on a network or the internet (e.g., **GitHub, GitLab, or Azure DevOps**) to facilitate team sharing.

Working Directory	$\xrightarrow{\text{git add}}$	**Staging Area**	$\xrightarrow{\text{git commit}}$
Your files		Prepared changes	

$\xrightarrow{\text{git commit}}$	**Local Repository**	$\xrightarrow{\text{git push}}$
Your .git folder		Remote (GitHub, etc.)

Figure 2: Git Workflow Stages

2.2 The Standard Workflow

- **Step 1:** Modify files in the working directory.
- **Step 2: Stage** the changes using `git add`.
- **Step 3: Commit** the staged changes to the local repository using `git commit` to create a permanent snapshot.
- **Step 4: Push** the local commits to the remote repository using `git push` so others can see them.
- **Step 5: Pull** changes from others into your local copy using `git pull`.

Example: Basic Git Commands

```
# Navigate to your TwinCAT project directory
cd C:\TwinCAT\Projects\MyMachineProject

# Initialize a new Git repository (only needed once)
git init

# Check the status of your files
git status

# Stage all changed files for commit
# The "." means "all files in current directory and subdirectories"
git add .

# Alternatively, stage specific files only
git add POUs/FB_Motor.TcPOU
git add POUs/MAIN.TcPOU

# Commit the staged changes with a descriptive message
```

```
# Good commit messages explain WHAT changed and WHY
git commit -m "Add motor control function block with safety interlock
    "

# Push commits to the remote repository (e.g., GitHub)
# 'origin' is the default name for the remote server
# 'main' is the branch name (formerly 'master')
git push origin main

# Pull latest changes from the remote repository
# Always do this before starting work to get teammates' changes
git pull origin main
```

```
# Additional useful Git commands for daily work

# View commit history
git log --oneline

# Example output:
# a1b2c3d Add emergency stop logic
# e4f5g6h Fix conveyor speed calculation
# i7j8k9l Initial project setup

# See what changed in a specific commit
git show a1b2c3d

# Create a new branch for a feature
git checkout -b feature/new-safety-system

# Switch back to main branch
git checkout main

# Merge feature branch into main
git merge feature/new-safety-system

# Discard local changes to a file (careful!)
git checkout -- POUs/MAIN.TcPOU

# See differences between working directory and last commit
git diff
```

Tip: Write meaningful commit messages! Instead of "fixed bug" or "updated code," write "Fix motor overtemperature alarm threshold from 80C to 85C per customer request #1234". This makes history searchable and useful.

3 TwinCAT 3 Specifics

Beckhoff's TwinCAT 3 is highly compatible with Git because its development environment (TcXaeShell) is based on **Visual Studio**, which has built-in Git integration. Unlike many PLC vendors who use expensive, proprietary binary formats, Beckhoff stores code in **human-readable XML**, which is ideal for version control.

3.1 The .gitignore File

You should not track every file in a project folder. Compiling code generates **software artifacts** (executables, object files, binaries) that do not belong in version control. A `.gitignore` file tells Git to ignore these files.

Example: TwinCAT .gitignore Template

```
# ==========================================
# TwinCAT 3 .gitignore Template
# Place this file in your project root folder
# ==========================================

# ------------------------------------------
# TwinCAT Specific Files
# ------------------------------------------

# Compiled library files (generated during build)
*.compiled-library

# TwinCAT boot project files
*.bootdata
*.bootdata-*

# Line ID files (when using "Separate LineIDs" setting)
*.TcLIds

# TwinCAT project cache
_Boot/
_CompileInfo/
_Libraries/

# TwinCAT log files
*.log
*.tpzip

# Binary/runtime files
*.tmc
*.tmcRefac

# ------------------------------------------
# Visual Studio Specific Files
# ------------------------------------------

# User-specific solution options (window layouts, etc.)
*.suo
*.user

# Build results
[Bb]in/
[Oo]bj/
[Dd]ebug/
[Rr]elease/

# Visual Studio cache/options directory
.vs/

# NuGet packages (can be restored)
```

```
packages/

# Resharper
_ReSharper*/
*.DotSettings.user

# ----------------------------------------
# Operating System Files
# ----------------------------------------

# Windows
Thumbs.db
ehthumbs.db
Desktop.ini

# macOS
.DS_Store
.AppleDouble
.LSOverride

# ----------------------------------------
# Backup Files
# ----------------------------------------

# Common backup extensions
*.bak
*.tmp
*~
*.orig

# TwinCAT backup files
*.TcPOU.bak
*.TcDUT.bak
*.TcGVL.bak
```

Note: Create the `.gitignore` file **before** your first commit. If you accidentally commit files that should be ignored, they will remain in the repository history even after adding them to `.gitignore`. You would need to use `git rm -cached` to remove them.

3.2 Essential Settings for Clean Code History

To make code reviews and comparisons ("diffs") easier, you must change several default TwinCAT settings to prevent "noise" in your XML files.

3.2.1 1. Separate LineIDs

By default, TwinCAT embeds "LineIDs" directly into the source code for online tracking. This creates constant, unnecessary changes in your Git history.

Action: Go to *Tools* > *Options* > *TwinCAT* > *PLC Environment* > *Write Options* and set **"Separate LineIDs" to True**.

3.2.2 2. Independent Project Files

Instead of one giant "mega-blob" file for the whole project, TwinCAT can store I/O devices and EtherCAT configurations in separate files.

Action: Go to *TwinCAT* > *XAE Environment* > *File Settings* and set these to **True** to make code reviews much simpler.

Example: XML LineID Noise

```
<!-- BAD: LineIDs embedded in source code (default setting) -->
<!-- Every time you open/save the file, LineIDs may change! -->
<!-- This creates "noise" in Git history - changes that aren't real code changes -->

<TcPlcObject Version="1.1.0.1">
  <POU Name="MAIN" Id="{12345678-1234-1234-1234-123456789abc}">
    <Declaration><![CDATA[
PROGRAM MAIN
VAR
    bStart : BOOL;
    nCounter : INT;
END_VAR
    ]]></Declaration>
    <Implementation>
      <ST><![CDATA[
<!-- LineIds are embedded directly in the code block -->
<LineId Id="1">IF bStart THEN</LineId>
<LineId Id="2">    nCounter := nCounter + 1;</LineId>
<LineId Id="3">END_IF</LineId>
      ]]></ST>
    </Implementation>
  </POU>
</TcPlcObject>
```

```
<!-- GOOD: With "Separate LineIDs" = True -->
<!-- Source code is clean and readable -->
<!-- LineIDs are stored in a separate .TcLIds file (which is in .gitignore) -->

<TcPlcObject Version="1.1.0.1">
  <POU Name="MAIN" Id="{12345678-1234-1234-1234-123456789abc}">
    <Declaration><![CDATA[
PROGRAM MAIN
VAR
    bStart : BOOL;
    nCounter : INT;
END_VAR
    ]]></Declaration>
    <Implementation>
      <ST><![CDATA[
IF bStart THEN
    nCounter := nCounter + 1;
END_IF
      ]]></ST>
    </Implementation>
  </POU>
</TcPlcObject>

<!-- The LineIDs are now in a separate file: MAIN.TcLIds -->
<!-- This file is listed in .gitignore and not tracked -->
```

Warning: Without the "Separate LineIDs" setting, your Git history will show changes to LineID numbers even when you haven't modified any actual code. This makes it nearly impossible to review real code changes and defeats much of the purpose of version control.

Setting	Location	Value
Separate LineIDs	TwinCAT > PLC Environment > Write Options	True
Separate I/O	TwinCAT > XAE Environment > File Settings	True
Separate Routes	TwinCAT > XAE Environment > File Settings	True

Table 2: Recommended TwinCAT Settings for Version Control

4 Visual Comparisons (Diffing)

Because TwinCAT code is XML-based, standard Git comparison tools look like raw text. To see differences in a user-friendly way (including graphical languages like Ladder), you should use the **TwinCAT Project Compare** tool.

- **Configuration:** You can configure Visual Studio to automatically launch `TcProjectCompare` whenever you want to see the difference between two commits.
- **Capabilities:** This tool works for Structured Text, I/O configurations, and even graphical programming languages.

```
# Configure Git to use TwinCAT Project Compare as diff tool
# Run these commands once to set up your Git configuration

# Set TcProjectCompare as the diff tool
git config --global diff.tool TcProjectCompare
git config --global difftool.TcProjectCompare.cmd \
    '"C:/TwinCAT/3.1/Components/TcProjectCompare/TcProjectCompare.exe" "$LOCAL" "$REMOTE"'

# Set TcProjectCompare as the merge tool
git config --global merge.tool TcProjectCompare
git config --global mergetool.TcProjectCompare.cmd \
    '"C:/TwinCAT/3.1/Components/TcProjectCompare/TcProjectCompare.exe" "$LOCAL" "$REMOTE" "$MERGED"'

# Now you can use these commands:
git difftool                    # Compare working directory with last commit
git difftool HEAD~1             # Compare with previous commit
git difftool branch1 branch2    # Compare two branches
```

Tip: The TwinCAT Project Compare tool can show side-by-side differences for Structured Text, Function Block Diagrams, Ladder Logic, and even hardware configurations. This is much more useful than looking at raw XML differences!

5 Conclusion

Using version control is **100% free** in TwinCAT 3. By using standard tools like Git and GitHub, automation engineers can adopt the same high-quality workflows used in the traditional software industry, leading to safer, more maintainable projects.

5.1 Summary of Best Practices

Practice	Recommendation
Version Control System	Use Git (industry standard)
Remote Repository	GitHub, GitLab, or Azure DevOps
.gitignore	Create before first commit
Separate LineIDs	Enable in TwinCAT settings
Commit Messages	Descriptive and meaningful
Branching	Use feature branches for new work
Code Review	Use TcProjectCompare for diffs

Table 3: Version Control Best Practices for TwinCAT

5.2 Quick Start Checklist

1. Install Git from https://git-scm.com
2. Create account on GitHub/GitLab/Azure DevOps
3. Configure TwinCAT settings (Separate LineIDs, etc.)
4. Create `.gitignore` file in project folder
5. Initialize repository: `git init`
6. Make first commit: `git add .` then `git commit -m "Initial commit"`
7. Create remote repository and push: `git push origin main`
8. Configure TcProjectCompare as diff tool

Quick Reference Card

Essential Git Commands

TwinCAT Files to Ignore

Last Updated: March 14, 2026

Command	Purpose
`git init`	Initialize new repository
`git clone <url>`	Copy remote repository
`git status`	Show changed files
`git add .`	Stage all changes
`git commit -m "msg"`	Create snapshot
`git push origin main`	Upload to remote
`git pull origin main`	Download from remote
`git log -oneline`	View history
`git diff`	Show unstaged changes
`git checkout -b <name>`	Create new branch

Table 4: Essential Git Commands

Pattern	Description
`*.compiled-library`	Compiled libraries
`*.TcLIds`	LineID files
`*.suo, *.user`	VS user settings
`_Boot/`	Boot project folder
`.vs/`	VS cache folder
`*.log`	Log files

Table 5: Common TwinCAT .gitignore Entries